SHOPKART E-COMMERCE BUSINESS ANALYTICS

SQL Pro Project

Student Name: Shubham Kumar
Course: B.Tech CSE (Data Analytics)
University: SAGE University, Bhopal
Submission Date: 31st March 2026

Table of Contents

1. Project Overview
2. Objectives
3. Database Structure
4. SQL Concepts Used
5. Key Business Queries
6. Insights
7. Challenges Faced
8. Solutions & Approach
9. Learning Outcomes
10. Future Improvements
11. Output Screenshots
12. Conclusion

---

1. Project Overview

This project focuses on analyzing e-commerce business data from ShopKart using SQL. The objective is to extract meaningful insights about customers, products, orders, and overall business performance.

The dataset includes multiple tables such as **Customers**, **Orders**, **Products**, **Categories**, and **Order_Items**.

---

2. Objectives

- Perform data analysis using SQL queries
- Understand customer purchasing behavior
- Identify top-performing products and categories
- Analyze revenue, profit, and order trends
- Solve real-world business problems using SQL

---

3. Database Structure

The project consists of the following tables:

- **Customers**
- **Orders**
- **Products**
- **Categories**
- **Order_Items**

These tables are connected using **primary and foreign keys**.

---

4. SQL Concepts Used

- SELECT, WHERE
- ORDER BY, LIMIT
- GROUP BY, HAVING
- INNER JOIN
- AGGREGATE FUNCTIONS (SUM, COUNT, AVG, MAX)
- SUBQUERIES

---

5. Key Business Queries

Some important business questions solved in this project include:

- Customers from a specific city
- Top expensive products
- Latest orders
- Total orders and quantities
- Revenue generated by city
- Most profitable product
- Repeat customers

- Cancelled order percentage
- Monthly order trends

---

6. Insights

- Identified top-performing products based on sales
- Found the most profitable categories
- Analyzed customer purchasing patterns
- Determined revenue contribution by city
- Evaluated order trends over time

---

7. Challenges Faced

While working on this project, I faced several challenges:

- Understanding complex joins between multiple tables
- Writing accurate aggregation queries
- Handling grouping and filtering conditions
- Avoiding logical mistakes in subqueries
- Interpreting business requirements into SQL queries

---

8. Solutions & Approach

To overcome these challenges:

- Broke down complex queries into smaller steps
- Practiced SQL concepts step-by-step
- Verified outputs using query execution
- Focused on understanding logic before coding
- Improved query structure for better readability

---

9. Learning Outcomes

- Gained strong understanding of SQL fundamentals
- Learned how to solve real-world business problems
- Improved logical thinking and analytical skills
- Gained hands-on experience in data analysis

10. Future Improvements

- Create dashboards using Power BI or Tableau
- Work with larger datasets
- Optimize SQL queries
- Perform advanced analytics

11. Output Screenshots

```sql
# PROJECT: SHOPKART E-COMMERCE BUSINESS ANALYTICS
# NAME: Shubham Kumar
# COURSE: B.Tech CSE (D.A)
# DEPARTMENT: SAGE UNIVERSITY, BHOPAL
# DESCRIPTION: Complete SQL case study covering database creation,
# joins, aggregations, profit analytics and business insights

-----------------------------------------------------

# CREATE DATABASE
create database shopkart_db;

# USE DATABASE
use shopkart_db;

-----------------------------------------------------
```

```sql
# CREATE TABLES

# CUSTOMERS TABLE
create table customers (
    customer_id int primary key,
    customer_name varchar(100),
    gender varchar(10),
    city varchar(50),
    signup_date date
);

# CATEGORIES TABLE
create table categories (
    category_id int primary key,
    category_name varchar(50)
);
```

```sql

# PRODUCTS TABLE
create table products (
    product_id int primary key,
    product_name varchar(100),
    category_id int,
    price decimal(10,2),
    cost decimal(10,2),
    foreign key (category_id) references categories(category_id)
);

# ORDERS TABLE
create table orders (
    order_id int primary key,
    customer_id int,
    order_date date,
    city varchar(50),
    order_status varchar(20),
    foreign key (customer_id) references customers(customer_id)
);

# ORDER_ITEMS TABLE
create table order_items (
    order_item_id int primary key,
    order_id int,
    product_id int,
    quantity int,
    foreign key (order_id) references orders(order_id),
    foreign key (product_id) references products(product_id)
);

-------------------------------------------------------

# INSERT DATA

# CUSTOMERS DATA
insert into customers values
(1,'Rahul Sharma','Male','Delhi','2022-01-10'),
(2,'Priya Mehta','Female','Mumbai','2021-05-20'),
(3,'Amit Patel','Male','Ahmedabad','2023-02-14'),
(4,'Sneha Reddy','Female','Hyderabad','2022-11-01'),
(5,'Karan Verma','Male','Delhi','2020-08-18'),
(6,'Neha Nair','Female','Chennai','2021-09-09'),
(7,'Rohit Gupta','Male','Pune','2023-03-12'),
(8,'Anjali Singh','Female','Bangalore','2022-06-25');

# CATEGORIES DATA
insert into categories values
(1,'Electronics'),
(2,'Fashion'),
(3,'Home Appliances');
```

```sql
# PRODUCTS DATA
insert into products values
(101,'Laptop',1,80000,65000),
(102,'Mobile',1,30000,22000),
(103,'Headphones',1,2000,1200),
(104,'T-Shirt',2,800,300),
(105,'Jeans',2,2000,900),
(106,'Microwave',3,7000,5000),
(107,'Refrigerator',3,30000,24000),
(108,'Air Conditioner',3,45000,35000);
```

```sql
# ORDERS DATA
insert into orders values
(1001,1,'2024-01-10','Delhi','Delivered'),
(1002,2,'2024-01-11','Mumbai','Delivered'),
(1003,3,'2024-01-12','Ahmedabad','Cancelled'),
(1004,4,'2024-01-13','Hyderabad','Delivered'),
(1005,5,'2024-01-15','Delhi','Delivered'),
(1006,6,'2024-01-16','Chennai','Pending'),
(1007,1,'2024-02-02','Delhi','Delivered'),
(1008,7,'2024-02-05','Pune','Delivered'),
(1009,8,'2024-02-08','Bangalore','Delivered'),
(1010,2,'2024-02-10','Mumbai','Cancelled');
```

```sql
# ORDER ITEMS DATA
insert into order_items values
(1,1001,101,1),
(2,1001,103,2),
(3,1002,102,1),
(4,1003,104,3),
(5,1004,106,1),
(6,1005,105,2),
(7,1005,104,1),
(8,1006,102,1),
(9,1007,108,1),
(10,1008,103,3),
(11,1009,107,1),
(12,1010,104,2);

-----------------------------------------------------
```

```
# LEVEL 1 - BASIC QUERIES

# Q1 CUSTOMERS FROM DELHI
select *
from customers
where city = 'Delhi';
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	customer_name	gender	city	signup_date
1	Rahul Sharma	Male	Delhi	2022-01-10
5	Karan Verma	Male	Delhi	2020-08-18
NULL	NULL	NULL	NULL	NULL

```
# Q2 LATEST 5 ORDERS
select *
from orders
order by order_date desc
limit 5;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

order_id	customer_id	order_date	city	order_status
1010	2	2024-02-10	Mumbai	Cancelled
1009	8	2024-02-08	Bangalore	Delivered
1008	7	2024-02-05	Pune	Delivered
1007	1	2024-02-02	Delhi	Delivered
1006	6	2024-01-16	Chennai	Pending
NULL	NULL	NULL	NULL	NULL

```
# Q3 TOP 3 EXPENSIVE PRODUCTS
select *
from products
order by price desc
limit 3;

-----------------------------------------------------
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

product_id	product_name	category_id	price	cost
101	Laptop	1	80000.00	65000.00
108	Air Conditioner	3	45000.00	35000.00
102	Mobile	1	30000.00	22000.00
NULL	NULL	NULL	NULL	NULL

```
# LEVEL 2 - AGGREGATION

# Q4 TOTAL NUMBER OF ORDERS
select count(*) as total_orders
from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_orders
10

```
# Q5 TOTAL QUANTITY SOLD PER PRODUCT
select product_id, sum(quantity) as total_quantity
from order_items
group by product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	total_quantity
101	1
102	2
103	5
104	6
105	2
106	1
107	1
108	1

```
# Q6 CITIES WITH MORE THAN 2 ORDERS
select city, count(*) as total_orders
from orders
group by city
having count(*) > 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	total_orders
Delhi	3

```
-----------------------------------------------------

# LEVEL 3 - JOINS

# Q7 CUSTOMER NAME WITH ORDER ID
select c.customer_name, o.order_id
from customers c
join orders o
on c.customer_id = o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	order_id
Rahul Sharma	1001
Rahul Sharma	1007
Priya Mehta	1002
Priya Mehta	1010
Amit Patel	1003
Sneha Reddy	1004
Karan Verma	1005
Neha Nair	1006
Rohit Gupta	1008
Anjali Singh	1009

```
# Q8 PRODUCT NAME IN EACH ORDER
select o.order_id, p.product_name
from orders o
join order_items oi
on o.order_id = oi.order_id
join products p
on oi.product_id = p.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	product_name
1001	Laptop
1002	Mobile
1006	Mobile
1001	Headphones
1008	Headphones
1003	T-Shirt
1005	T-Shirt
1010	T-Shirt
1005	Jeans
1004	Microwave
1009	Refrigerator
1007	Air Conditioner

```
# Q9 TOTAL QUANTITY PER CUSTOMER
select c.customer_name, sum(oi.quantity) as total_quantity
from customers c
join orders o
on c.customer_id = o.customer_id
join order_items oi
on o.order_id = oi.order_id
group by c.customer_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_name	total_quantity
Rahul Sharma	4
Priya Mehta	3
Amit Patel	3
Sneha Reddy	1
Karan Verma	3
Neha Nair	1
Rohit Gupta	3
Anjali Singh	1

```
---------------------------------------------------

# LEVEL 4 - PROFIT ANALYTICS

# Q10 PROFIT PER PRODUCT
select product_name, (price - cost) as profit_per_unit
from products;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	profit_per_unit
Laptop	15000.00
Mobile	8000.00
Headphones	800.00
T-Shirt	500.00
Jeans	1100.00
Microwave	2000.00
Refrigerator	6000.00
Air Conditioner	10000.00

```
# Q11 CATEGORY-WISE TOTAL PROFIT
select cat.category_name,
sum((p.price - p.cost) * oi.quantity) as total_profit
from order_items oi
join products p
on oi.product_id = p.product_id
join categories cat
on p.category_id = cat.category_id
group by cat.category_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_name	total_profit
Electronics	35000.00
Fashion	5200.00
Home Appliances	18000.00

```
---------------------------------------------------

# LEVEL 5 - CUSTOMER INTELLIGENCE

# Q12 CUSTOMERS WITH MORE THAN 1 ORDER
select customer_id, count(*) as total_orders
from orders
group by customer_id
having count(*) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_orders
1	2
2	2

```
# Q13 CUSTOMERS ABOVE AVERAGE PURCHASE QUANTITY
select customer_id, sum(quantity) as total_quantity
from orders o
join order_items oi
on o.order_id = oi.order_id
group by customer_id
having sum(quantity) > (
    select avg(quantity) from order_items
);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_quantity
1	4
2	3
3	3
5	3
7	3

```
--------------------------------------------------

# LEVEL 6 - BUSINESS INSIGHTS

# Q14 CITY WITH HIGHEST REVENUE
select o.city,
sum(p.price * oi.quantity) as revenue
from orders o
join order_items oi
on o.order_id = oi.order_id
join products p
on oi.product_id = p.product_id
group by o.city
order by revenue desc
limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

city	revenue
Delhi	133800.00

```
# Q15 MOST PROFITABLE PRODUCT
select p.product_name,
sum((p.price - p.cost) * oi.quantity) as total_profit
from order_items oi
join products p
on oi.product_id = p.product_id
group by p.product_name
order by total_profit desc
limit 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

product_name	total_profit
Mobile	16000.00

```
# Q16 MONTHLY ORDER TREND
select month(order_date) as month,
count(*) as total_orders
from orders
group by month(order_date);

--------------------------------------------------
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	total_orders
1	6
2	4

```
# CAPSTONE - BUSINESS ANALYSIS

# TOP 3 CUSTOMERS BY PURCHASE
select c.customer_name, sum(oi.quantity) as total_purchase
from customers c
join orders o
on c.customer_id = o.customer_id
join order_items oi
on o.order_id = oi.order_id
group by c.customer_name
order by total_purchase desc
limit 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

customer_name	total_purchase
Rahul Sharma	4
Priya Mehta	3
Amit Patel	3

```
# REVENUE PER CITY
select o.city, sum(p.price * oi.quantity) as revenue
from orders o
join order_items oi
on o.order_id = oi.order_id
join products p
on oi.product_id = p.product_id
group by o.city;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

city	revenue
Delhi	133800.00
Mumbai	31600.00
Chennai	30000.00
Pune	6000.00
Ahmedabad	2400.00
Hyderabad	7000.00
Bangalore	30000.00

```sql
# PROFIT PER CATEGORY
select cat.category_name,
sum((p.price - p.cost) * oi.quantity) as profit
from order_items oi
join products p
on oi.product_id = p.product_id
join categories cat
on p.category_id = cat.category_id
group by cat.category_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_name	profit
Electronics	35000.00
Fashion	5200.00
Home Appliances	18000.00

```sql
# REPEAT CUSTOMERS COUNT
select count(*) as repeat_customers
from (
    select customer_id
    from orders
    group by customer_id
    having count(*) > 1
) as t;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

repeat_customers
2

```sql
# CANCELLED ORDER PERCENTAGE
select
(count(case when order_status = 'Cancelled' then 1 end) * 100.0 / count(*)) as cancel_percentage
from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cancel_percentage
20.00000

```sql
--------------------------------------------------

# PROJECT COMPLETED
# all required sql queries, analysis, and business insights have been executed successfully.

# author: shubham kumar
# project: shopkart e-commerce analytics
# status: completed
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	01:16:15	use shopkart_db	0 row(s) affected	0.000 sec
2	01:17:20	create table customers (customer_id int primary key, customer_name varchar(100), gender varchar(10),...	0 row(s) affected	0.078 sec
3	01:17:25	create table categories (category_id int primary key, category_name varchar(50))	0 row(s) affected	0.062 sec
4	01:18:06	create table products (product_id int primary key, product_name varchar(100), category_id int, price ...	0 row(s) affected	0.093 sec
5	01:18:10	create table orders (order_id int primary key, customer_id int, order_date date, city varchar(50), or...	0 row(s) affected	0.063 sec
6	01:18:55	create table order_items (order_item_id int primary key, order_id int, product_id int, quantity int, for...	0 row(s) affected	0.078 sec
7	01:19:36	insert into customers values (1,'Rahul Sharma','Male','Delhi','2022-01-10'), (2,'Priya Mehta','Female','Mumbai','20...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.031 sec
8	01:19:39	insert into categories values (1,'Electronics'), (2,'Fashion'), (3,'Home Appliances')	3 row(s) affected Records: 3 Duplicates: 0 Warnings: 0	0.015 sec
9	01:20:49	insert into products values (101,'Laptop',1,80000,65000), (102,'Mobile',1,30000,22000), (103,'Headphones',1,2...	8 row(s) affected Records: 8 Duplicates: 0 Warnings: 0	0.031 sec
10	01:21:39	insert into orders values (1001,1,'2024-01-10','Delhi','Delivered'), (1002,2,'2024-01-11','Mumbai','Delivered'), (10...	10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0	0.015 sec
11	01:22:22	insert into order_items values (1,1001,101,1), (2,1001,103,2), (3,1002,102,1), (4,1003,104,3), (5,1004,106,1), (...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	0.016 sec
12	01:22:59	select * from customers where city = 'Delhi' LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
13	01:23:34	select * from orders order by order_date desc limit 5	5 row(s) returned	0.015 sec / 0.000 sec
14	01:24:22	select * from products order by price desc limit 3	3 row(s) returned	0.015 sec / 0.000 sec
15	01:24:59	select count(*) as total_orders from orders LIMIT 0, 1000	1 row(s) returned	0.031 sec / 0.000 sec
16	01:25:31	select product_id, sum(quantity) as total_quantity from order_items group by product_id LIMIT 0, 1000	8 row(s) returned	0.015 sec / 0.000 sec
17	01:26:07	select city, count(*) as total_orders from orders group by city having count(*) > 2 LIMIT 0, 1000	1 row(s) returned	0.016 sec / 0.000 sec
18	01:26:37	select c.customer_name, o.order_id from customers c join orders o on c.customer_id = o.customer_id LIMIT 0, ...	10 row(s) returned	0.000 sec / 0.000 sec
19	01:26:52	select c.customer_name, o.order_id from customers c join orders o on c.customer_id = o.customer_id LIMIT 0, ...	10 row(s) returned	0.000 sec / 0.000 sec
20	01:27:16	select o.order_id, p.product_name from orders o join order_items oi on o.order_id = oi.order_id join products p ...	12 row(s) returned	0.000 sec / 0.000 sec
20	01:27:16	select o.order_id, p.product_name from orders o join order_items oi on o.order_id = oi.order_id join products p ...	12 row(s) returned	0.000 sec / 0.000 sec
21	01:28:06	select c.customer_name, sum(oi.quantity) as total_quantity from customers c join orders o on c.customer_id = o...	8 row(s) returned	0.000 sec / 0.000 sec
22	01:28:34	select product_name, (price - cost) as profit_per_unit from products LIMIT 0, 1000	8 row(s) returned	0.016 sec / 0.000 sec
23	01:29:02	select cat.category_name, sum((p.price - p.cost) * oi.quantity) as total_profit from order_items oi join products p ...	3 row(s) returned	0.016 sec / 0.000 sec
24	01:29:28	select customer_id, count(*) as total_orders from orders group by customer_id having count(*) > 1 LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
25	01:29:57	select customer_id, sum(quantity) as total_quantity from orders o join order_items oi on o.order_id = oi.order_id ...	5 row(s) returned	0.031 sec / 0.000 sec
26	01:30:42	select o.city, sum(p.price * oi.quantity) as revenue from orders o join order_items oi on o.order_id = oi.order_id j...	1 row(s) returned	0.016 sec / 0.000 sec
27	01:31:07	select p.product_name, sum((p.price - p.cost) * oi.quantity) as total_profit from order_items oi join products p on...	1 row(s) returned	0.016 sec / 0.000 sec
28	01:31:14	select p.product_name, sum((p.price - p.cost) * oi.quantity) as total_profit from order_items oi join products p on...	1 row(s) returned	0.000 sec / 0.000 sec
29	01:31:50	select month(order_date) as month, count(*) as total_orders from orders group by month(order_date) LIMIT 0, 1...	2 row(s) returned	0.031 sec / 0.000 sec
30	01:32:20	select c.customer_name, sum(oi.quantity) as total_purchase from customers c join orders o on c.customer_id =...	3 row(s) returned	0.000 sec / 0.000 sec
31	01:32:53	select o.city, sum(p.price * oi.quantity) as revenue from orders o join order_items oi on o.order_id = oi.order_id j...	7 row(s) returned	0.015 sec / 0.000 sec
32	01:33:00	select o.city, sum(p.price * oi.quantity) as revenue from orders o join order_items oi on o.order_id = oi.order_id j...	7 row(s) returned	0.015 sec / 0.000 sec
33	01:33:24	select cat.category_name, sum((p.price - p.cost) * oi.quantity) as profit from order_items oi join products p on oi....	3 row(s) returned	0.016 sec / 0.000 sec
34	01:33:50	select count(*) as repeat_customers from (select customer_id from orders group by customer_id havi...	1 row(s) returned	0.000 sec / 0.000 sec
35	01:34:25	select (count(case when order_status = 'Cancelled' then 1 end) * 100.0 / count(*)) as cancel_percentage from...	1 row(s) returned	0.031 sec / 0.000 sec

---

12. Conclusion

This project helped me understand how SQL can be used for business analytics.
I successfully executed all required queries and generated meaningful insights from the dataset.

Project Status:

- Completed Successfully
- All Queries Executed
- Outputs Verified